UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA

MANEJO E IMPLEMENTACIÓN DE ARCHIVOS 1

LABORATORIO — PROYECTO 1

ExtreamFS

Nombre: Yury Urbano Santos Chagil
Carnet: 202113318
Semestre: 1S2026
Fecha: 18 de Marzo de 2026

INTRODUCCIÓN

El presente manual técnico describe la arquitectura, funcionamiento, estructura interna, compilación, ejecución e integración del proyecto ExtreamFS, desarrollado para el curso de Manejo e Implementación de Archivos. El sistema implementa un conjunto de comandos para la administración de discos, particiones, formateo, usuarios, grupos, carpetas, archivos y reportes, integrando un backend en C++ con un frontend web en Angular.

El proyecto fue diseñado con una arquitectura modular que separa la lógica del negocio del mecanismo de interacción con el usuario. La lógica principal se encuentra implementada en el backend, mientras que el frontend actúa como una capa de presentación desde la cual es posible ingresar comandos, cargar scripts y visualizar la salida del sistema.

OBJETIVOS

El objetivo principal del proyecto es proporcionar una herramienta capaz de simular operaciones de bajo nivel sobre discos y sistemas de archivos, incluyendo:

creación y eliminación de discos

creación y administración de particiones

montaje de particiones

formateo en EXT2

manejo de usuarios y grupos

creación de carpetas y archivos

lectura de archivos

generación de reportes estructurales

ejecución de comandos desde una interfaz web

ARQUITECTURA DEL SISTEMA

El proyecto se compone de dos grandes capas:

Backend

El backend fue implementado en C++ y contiene toda la lógica principal del proyecto. Esta capa se encarga de:

- analizar comandos
- validar parámetros
- ejecutar operaciones sobre discos
- administrar el sistema de archivos
- manejar sesiones
- generar reportes
- responder solicitudes del frontend por medio de una API HTTP

Frontend

El frontend fue implementado en Angular y tiene como objetivo brindar una interfaz visual para:

- ingresar comandos manualmente
- cargar archivos de script
- ejecutar instrucciones contra el backend
- visualizar la salida generada
- comprobar el estado de conexión con el backend

Comunicación entre frontend y backend

La comunicación entre ambas capas se realiza mediante una API HTTP local, utilizando dos endpoints principales:

- `GET /api/health`
- `POST /api/execute`

El frontend consume estos endpoints utilizando `HttpClient`, y el backend responde en formato JSON.

Estructura del proyecto

La estructura general del proyecto se organiza en dos carpetas principales: `backend` y `frontend`.

Estructura general

```
ExtreamFS/
├── backend/
|   ├── build/
|   ├── include/
|   |   ├── api/
|   |   ├── core/
|   |   ├── disk/
|   |   ├── fs/
|   |   ├── reports/
|   |   └── third_party/
|   ├── src/
|   |   ├── api/
|   |   ├── core/
|   |   ├── disk/
|   |   ├── fs/
|   |   ├── reports/
|   |   └── main.cpp
|   └── CMakeLists.txt
|
└── frontend/
    └── extreamfs-frontend/
        ├── src/
        |   └── app/
        ├── angular.json
        ├── package.json
        └── ...
```

Descripción técnica del backend

`main.cpp`

El archivo `main.cpp` es el punto de entrada principal del sistema. Su función es determinar el modo en que se ejecutará el proyecto.

El sistema soporta dos modos:

1. **Modo CLI**
    - ejecuta el analizador en consola interactiva
    - permite escribir comandos manualmente en terminal
2. **Modo API**
    - inicia el servidor HTTP
    - permite recibir comandos desde el frontend Angular

Cuando se ejecuta con el parámetro `--api`, el programa levanta el servidor HTTP en el puerto `8080`.

`Analyzer`

La clase `Analyzer` es el núcleo del sistema. Se encarga de:

- recibir líneas de comandos
- identificar el comando principal
- parsear parámetros
- validar la sintaxis
- invocar el módulo correspondiente

Métodos relevantes

`RunInteractive()`

Ejecuta el analizador en modo interactivo desde consola.

`ExecuteScript(const std::string& input)`

Método agregado para la integración web. Recibe un bloque de texto, lo separa por líneas, procesa comentarios, ignora líneas vacías, ejecuta cada instrucción y devuelve toda la salida como un `std::string`.

Esto permitió reutilizar la lógica existente sin reescribir el sistema para el frontend.

Módulos del backend

a) Módulo `disk/`

Contiene la lógica relacionada con discos y particiones.

Funciones principales:

- `mkdisk`
- `rmdisk`
- `fdisk`
- `mount`
- `mounted`

b) Módulo `fs/`

Contiene la lógica del sistema de archivos.

Funciones principales:

- `mkfs`
- `login`
- `logout`
- `mkgrp`
- `rmgrp`
- `mkusr`
- `rmusr`
- `chgrp`
- `mkdir`
- `mkfile`
- `cat`

c) Módulo `reports/`

Contiene la generación de reportes del sistema.

Reportes implementados:

- `mbr`
- `disk`
- `inode`
- `block`
- `bm_inode`

- `bm_block`
- `sb`
- `file`
- `ls`
- `tree`

`HttpServer`

Para integrar el backend con el frontend se implementó la clase `HttpServer`, la cual expone la lógica del sistema a través de una API local.

Se utilizó la librería **cpp-httplib** como servidor HTTP header-only.

Endpoint 1: estado del backend

Ruta: `GET /api/health`

Función:

- permitir al frontend verificar si el backend está encendido y accesible

Descripción técnica del backend

`main.cpp`

El archivo `main.cpp` es el punto de entrada principal del sistema. Su función es determinar el modo en que se ejecutará el proyecto.

El sistema soporta dos modos:

1. **Modo CLI**
    - ejecuta el analizador en consola interactiva
    - permite escribir comandos manualmente en terminal
2. **Modo API**
    - inicia el servidor HTTP
    - permite recibir comandos desde el frontend Angular

Cuando se ejecuta con el parámetro `--api`, el programa levanta el servidor HTTP en el puerto `8080`.

`Analyzer`

La clase `Analyzer` es el núcleo del sistema. Se encarga de:

- recibir líneas de comandos
- identificar el comando principal
- parsear parámetros
- validar la sintaxis
- invocar el módulo correspondiente

Métodos relevantes

`RunInteractive()`

Ejecuta el analizador en modo interactivo desde consola.

`ExecuteScript(const std::string& input)`

Método agregado para la integración web. Recibe un bloque de texto, lo separa por líneas, procesa comentarios, ignora líneas vacías, ejecuta cada instrucción y devuelve toda la salida como un `std::string`.

Esto permitió reutilizar la lógica existente sin reescribir el sistema para el frontend.

Módulos del backend

a) Módulo `disk/`

Contiene la lógica relacionada con discos y particiones.

Funciones principales:

- `mkdisk`
- `rmdisk`
- `fdisk`
- `mount`
- `mounted`

b) Módulo `fs/`

Contiene la lógica del sistema de archivos.

Funciones principales:

- `mkfs`
- `login`
- `logout`
- `mkgrp`
- `rmgrp`

- `mkusr`
- `rmusr`
- `chgrp`
- `mkdir`
- `mkfile`
- `cat`

c) Módulo `reports/`

Contiene la generación de reportes del sistema.

Reportes implementados:

- `mbr`
- `disk`
- `inode`
- `block`
- `bm_inode`
- `bm_block`
- `sb`
- `file`
- `ls`
- `tree`

`HttpServer`

Para integrar el backend con el frontend se implementó la clase `HttpServer`, la cual expone la lógica del sistema a través de una API local.

Se utilizó la librería **cpp-httplib** como servidor HTTP header-only.

Endpoint 1: estado del backend

Ruta: `GET /api/health`

Función:

- permitir al frontend verificar si el backend está encendido y accesible

Endpoint 2: ejecución de comandos

Ruta: `POST /api/execute`

Función:

- recibir un bloque de comandos desde el frontend
- procesarlo con `Analyzer::ExecuteScript`
- devolver la salida en formato JSON

stado en memoria del backend

El backend mantiene información en memoria mientras el proceso siga ejecutándose, especialmente en operaciones como:

- montaje de particiones
- sesión activa
- referencias temporales de ejecución

Esto significa que, para correr scripts de evaluación con IDs esperados como `181A`, `182A`, `181B` y `181C`, es recomendable iniciar el backend en estado limpio.

Descripción técnica del frontend

Tecnologías utilizadas

El frontend fue desarrollado con:

- Angular
- TypeScript
- HTML
- CSS

Pantalla principal

La pantalla principal del frontend incluye:

- encabezado del sistema
- indicador de estado del backend
- área de entrada de comandos
- selector de archivo para cargar scripts
- botón de ejecución
- botón de limpieza
- consola de salida
- panel de reportes

`app.component.ts`

Este archivo contiene la lógica principal del frontend.

Funciones implementadas:

- almacenar el texto del área de entrada
- leer el archivo cargado desde el selector
- invocar el backend con `POST /api/execute`
- consultar el estado del backend con `GET /api/health`
- actualizar la consola de salida
- controlar el estado visual del sistema

`backend.service.ts`

Este servicio encapsula la comunicación con la API.

Funciones:

- `checkHealth()`
- `executeCommands(commands: string)`

El servicio utiliza `HttpClient` para enviar solicitudes al backend local en:

`http://localhost:8080/api`

`app.config.ts`

Este archivo registra los proveedores necesarios de Angular, incluyendo:

- `provideRouter`
- `provideHttpClient`
- `provideZoneChangeDetection`

Esto permitió habilitar la comunicación HTTP con el backend.

Flujo de funcionamiento del sistema

El flujo operativo del sistema es el siguiente:

1. El usuario abre la aplicación web.
2. El frontend consulta `GET /api/health`.
3. Si el backend está activo, el estado cambia a “Conectado”.
4. El usuario escribe comandos o carga un archivo de script.
5. El frontend envía los comandos a `POST /api/execute`.
6. El backend recibe el texto.
7. `HttpServer` pasa el contenido a `Analyzer`.
8. `Analyzer` ejecuta línea por línea.
9. Cada módulo procesa su instrucción correspondiente.
10. La salida se captura y se retorna como JSON.
11. El frontend presenta el resultado en la consola web.

Comandos implementados

El sistema implementa comandos distribuidos en varias áreas.

Comandos de discos y particiones

- `mkdisk`
- `rmdisk`
- `fdisk`
- `mount`
- `mounted`

Comandos del sistema de archivos

- `mkfs`
- `cat`

Comandos de administración de usuarios y grupos

- `login`
- `logout`
- `mkgrp`
- `rmgrp`
- `mkusr`
- `rmusr`
- `chgrp`

Comandos de archivos y carpetas

- `mkdir`
- `mkfile`

Comando de reportes

- `rep`

Validaciones implementadas

El sistema incorpora validaciones funcionales y sintácticas para garantizar robustez.

Entre ellas destacan:

- parámetros obligatorios
- detección de parámetros inválidos
- verificación de rutas
- control de tamaños negativos
- máximo de longitud para nombres de usuario y grupo
- control de sesión activa
- verificación de existencia de particiones
- verificación de espacio suficiente para particiones
- control de montaje duplicado
- control de grupo o usuario repetido
- validación del uso de `-p` en `mkdir`
- verificación de archivos y carpetas para reportes

Estas validaciones fueron comprobadas durante la ejecución del script final de prueba.

Proceso de compilación y ejecución

Compilación del backend

Desde la carpeta `backend` se compila con:

```
cmake --build build -j
```

Ejecución del backend en modo API

```
./build/extreamfs –api
```

Esto inicia el servidor en el puerto `8080`.

Ejecución del frontend

Desde la carpeta del proyecto Angular:

```
ng serve -o
```

Esto levanta el frontend normalmente en:

```
http://localhost:4200
```

Flujo recomendado de arranque

1. abrir una terminal para backend
2. compilar si es necesario
3. ejecutar `./build/extreamfs --api`
4. abrir otra terminal para frontend
5. ejecutar `ng serve -o`
6. abrir `http://localhost:4200`

Pruebas realizadas

Durante el desarrollo del sistema se validaron los siguientes escenarios:

- creación y eliminación de discos
- creación de particiones primarias, extendidas y lógicas
- montaje de particiones
- consulta de particiones montadas
- formateo EXT2
- inicio y cierre de sesión
- gestión de grupos
- gestión de usuarios
- modificación de grupo de usuario
- eliminación lógica de usuarios y grupos
- creación de carpetas con y sin `-p`
- creación de archivos por tamaño
- creación de archivos con contenido externo
- lectura de archivos
- generación de reportes estructurales
- ejecución de comandos desde el frontend
- verificación de errores esperados en script de prueba

Resultados obtenidos

El sistema logró integrar satisfactoriamente:

- lógica de negocio en C++
- ejecución por consola
- ejecución desde aplicación web
- comunicación mediante API HTTP
- captura y retorno de salida
- generación de reportes del sistema de archivos

La corrida final del script adaptado confirmó el correcto funcionamiento de la mayoría de comandos, incluyendo los casos de éxito y los errores intencionales definidos como parte del proceso de validación.

Consideraciones técnicas importantes

IDs de montaje

Los IDs generados por `mount` dependen del estado actual del backend. Por ello, para ejecutar scripts de calificación con IDs fijos, se recomienda reiniciar el backend y trabajar con una corrida limpia.

Persistencia temporal en memoria

Mientras el proceso del backend siga ejecutándose, ciertos estados permanecen activos, como:

- particiones montadas
- sesión actual

Scripts de prueba

Se recomienda distinguir entre:

- script de validación completa con errores intencionales
- script limpio de demostración

Esto facilita la exposición y la comprobación técnica ante evaluadores.

CONCLUSIONES

El proyecto ExtreamFS implementa satisfactoriamente una solución funcional para la administración simulada de discos y sistemas de archivos, integrando un backend robusto en C++ con un frontend moderno en Angular.

La arquitectura modular permitió reutilizar la lógica ya existente del analizador, añadir una capa HTTP sin alterar el núcleo del sistema y habilitar la interacción desde una aplicación web local. Las pruebas realizadas demostraron que el sistema responde correctamente ante operaciones válidas y también ante errores intencionales, validando así tanto la funcionalidad como la robustez del proyecto.